

List of Practical		
Sr. No	Program Title	CO Mapping
1.	Implementation of Linux Commands Introduction of Unix/Linux Operating system and their architecture Display system information using uname, hostname, and date etc. File operations using cat, touch, cp, mv, rm, and chmod ,umask etc. Create, view, and navigate directories using mkdir, rmdir, cd, pwd, ls etc. Disk Commands df, du, mount, unmount, mkfs, fsck etc. Use redirection and piping in commands File compression and archiving using tar, gzip, zip, unzip etc. Process commands ps,kill, killall,nice, pgrep, top,htop etc. Network commands ifconfig, ping, netstat, host,ip route etc. Administrator Commands Adduser, Passwd, deluser, usermod, groupadd etc	CO1
2.	Shell Scripting Programming Write a shell script to ask your name, program name and enrollment number and print it on the screen. Write a shell script to find the sum, the average and the product of the four integers entered. write shell script to find average of numbers given at command line Write a shell program to exchange the values of two variables Write a shell program to Print Numbers 1 to 10 using while & do while loop. Write a shell program to Print Numbers 1 to 10 using for loop. Write a shell script to display the digits which are in odd position in a given 5-digit number. Write a shell program to search for a given number from the list of numbers provided using binary search method.	CO1

	Write a shell program to concatenate two strings and find the length of the resultant string Write a shell script to find the smallest of three numbers Write a shell program to count number of words, characters, white spaces and special symbols in a given text	
	Process & Thread Management	
3.	Introduction to C Programming (Statement, Conditional Statement, Loop, Array & Function)	C02
4.	Implement FCFS CPU Scheduling algorithm.	C02
5.	Implement the SJF CPU Scheduling algorithm (For both Pre-emptive and non-pre-emptive).	C02
6.	Implement PRIORITY CPU Scheduling Algorithm (For both Pre-emptive and non-pre-emptive).	C02
7.	Implement Round-Robin CPU Scheduling Algorithm	C02
8.	Implement Multi-Level Queue CPU Scheduling algorithm.	C02
9.	Implement Multilevel Queue CPU Scheduling Algorithm.	C02
	Concurrency and Deadlock Management	
10.	Execute the RACE Condition of Process Synchronization.	C02
11.	Implement the Producer–consumer problem using semaphores.	C02
12.	Design a code and implement the Dining Philosopher problem.	C02
13.	Implement Banker’s algorithm of Deadlock Avoidance.	C02
14.	Execute an algorithm for Deadlock Detection.	C02
	Memory Management	
15.	Implement Contiguous memory variable size partition scheme.	C03
16.	Simulate the First-Fit contiguous memory allocation technique.	C03
17.	Simulate the Best-Fit contiguous memory allocation technique.	C03

18.	Simulate the Worst-Fit contiguous memory allocation technique.	C03
19.	Implement the Non-contiguous Memory Allocation by using Paging.	C03
20.	Implement Contiguous memory variable size partition scheme.	C03
	Page Replacement	
21.	Write a Program to simulate the FIFO page replacement algorithm.	C03
22.	Write a Program to simulate the LRU page replacement Algorithm.	C03
23.	Write a Program to simulate the Optimal page replacement Algorithm.	C03
	Disk Scheduling	C03
24.	Write a program to simulate FCFS Disk Scheduling Algorithm.	C03
25.	Write a Program to simulate the SSTF Disk Scheduling Algorithm.	C03
26.	Write a program to simulate SCAN Disk Scheduling Algorithm.	C03
27.	Write a Program to simulate the C SCAN Disk Scheduling Algorithm.	C03
28.	Write a Program to simulate the LOOK Disk Scheduling Algorithm.	C03
	Modern Operating System	C03
29.	Introduction of CUDA Programming.	C03
30.	Write a program in CUDA print message "Welcome CUDA programming"	C03
31.	Implement matrix multiplication using shared memory in CUDA.	C03

32.	Connects to VMware vCenter and lists all virtual machines along with their power state.	C03
33.	Create a new virtual machine in Azure with specified configurations.	C03
34.	Deploy a simple HTTP-triggered distributed Azure Function.	C03
Total Hours:		48
*Competitive coding list will be shared with the students.		

 NIET Greater Noida Autonomous Institute	NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY, GR. NOIDA	Session: 2025-26
		III Semester
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING		

1. Write a shell script to ask your name, program name and enrolment number and print it on the screen.

STEP 1 : Create a file named- nano userinfo.sh

STEP 2: Write the code and then, Press CTRL + X, then Y, and hit Enter to save and exit.

STEP 3: To execute the file type - chmod +x userinfo.sh

STEP 4: For output type- ./ userinfo.sh

CODE

```
echo "Enter your name:"
read name
echo "Enter your program name:"
read program
echo "Enter your enrollment number:"
read enroll
echo "Name: $name"
echo "Program: $program"
echo "Enrollment Number: $enroll"
```

```
vaishali@DESKTOP-LHT8EKU: x + v
echo "Enter your name:"
read name
echo "Enter your program name:"
read program
echo "Enter your enrollment number:"
read enroll
echo "Name: $name"
echo "Program: $program"
echo "Enrollment Number: $enroll"

"userinfo.sh" 10L, 207B 10,0-1 All
```

```
vaishali@DESKTOP-LHT8EKU: x + v
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)

 * Documentation: https://help.ubuntu.com
 * Management:   https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Jun 30 08:41:32 UTC 2025

System load: 0.23          Processes: 54
Usage of /: 0.2% of 1006.85GB Users logged in: 0
Memory usage: 5%          IPv4 address for eth0: 172.24.112.75
Swap usage: 0%

This message is shown once a day. To disable it please create the
/home/vaishali/.hushlogin file.
vaishali@DESKTOP-LHT8EKU:~$
vaishali@DESKTOP-LHT8EKU:~$ vi userinfo.sh
vaishali@DESKTOP-LHT8EKU:~$ chmod +x userinfo.sh
vaishali@DESKTOP-LHT8EKU:~$ ./userinfo.sh
Enter your name:
Vaishali
Enter your program name:
CSE Lab
Enter your enrollment number:
123
Name: Vaishali
Program: CSE Lab
Enrollment Number: 123
vaishali@DESKTOP-LHT8EKU:~$
```

2. Write a shell script Sum, average, and product of 4 integers

STEP 1 : Create a file named- vi mathcalcul2.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type chmod +x mathcalcul2.sh

STEP 4: For output type ./mathcalcul2.sh

```
#!/bin/bash
```

```
echo "Enter four integers:"
```

```
read a b c d
```

```
sum=$((a + b + c + d))
```

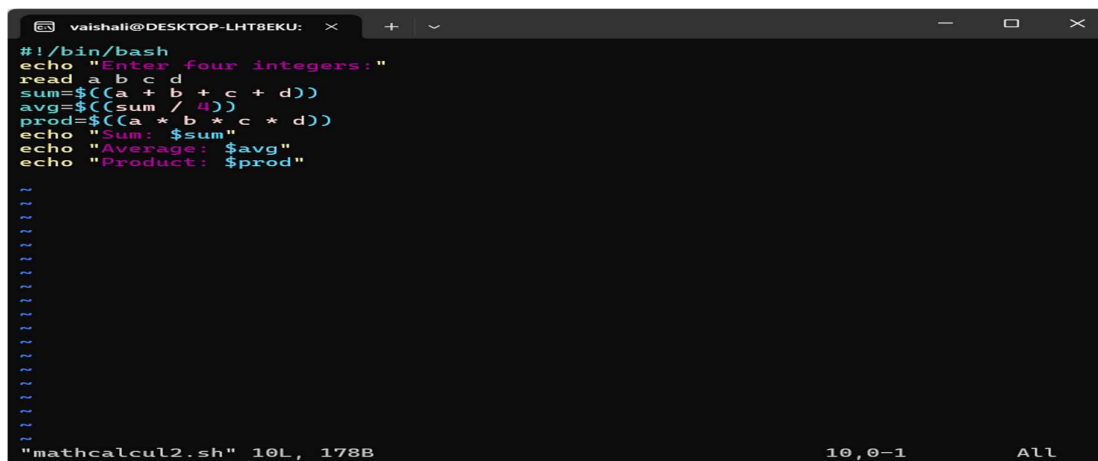
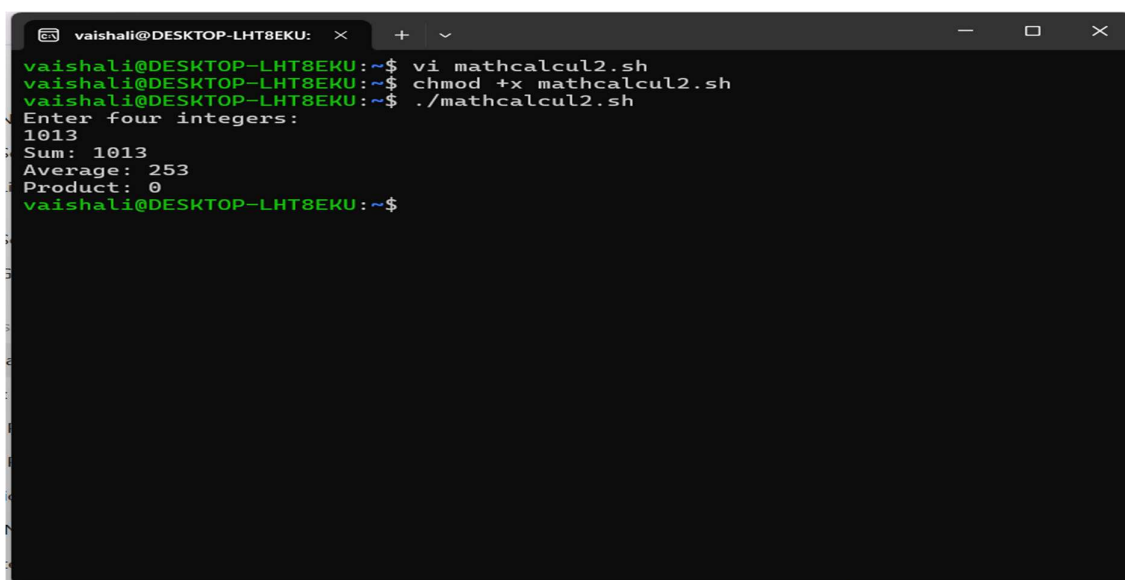
```
avg=$((sum / 4))
```

```
prod=$((a * b * c * d))
```

```
echo "Sum: $sum"
```

```
echo "Average: $avg"
```

```
echo "Product: $prod"
```

A terminal window titled 'vaishali@DESKTOP-LHT8EKU:' with a dark background. It displays the script content from the previous block, including the shebang, echo prompts, read command, arithmetic calculations for sum, average, and product, and the final echo statements. The window has standard Linux window controls (minimize, maximize, close) at the top right. At the bottom, it shows the filename 'mathcalcul2.sh' and line/byte counts '10L, 178B'.A terminal window titled 'vaishali@DESKTOP-LHT8EKU:' showing the execution of the script. The user enters 'vi mathcalcul2.sh', 'chmod +x mathcalcul2.sh', and './mathcalcul2.sh'. The script prompts 'Enter four integers:' and the user enters '1013'. The script then outputs 'Sum: 1013', 'Average: 253', and 'Product: 0'. The prompt returns to the user. The window has standard Linux window controls at the top right.

3. write shell script to find average of numbers given at command line

STEP 1 : Create a file named- vi avgcmd.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type chmod +x avgcmd.sh

STEP 4: For output type ./avgcmd.sh 10 20 30 40 (Since you didn't pass any numbers, \$# (which stores the number of arguments) was 0, leading to this line failing: avg=\$((sum / count)) # becomes avg=\$((0 / 0)))

```
#!/bin/bash

sum=0

count=$#

for num in "$@"
do
    sum=$((sum + num))
done

avg=$((sum / count))

echo "Average: $avg"
```



```
vaishali@DESKTOP-LHT8EKU: × + ∨
```

```
#!/bin/bash
sum=0
count=0
for num in "$@"
do
    sum=$((sum + num))
done
avg=$((sum / count))
echo "Average: $avg"
```

```
-- INSERT --
```

10,1 All

```
vaishali@DESKTOP-LHT8EKU: ~$ vi avgcmd.sh
vaishali@DESKTOP-LHT8EKU: ~$ chmod +x avgcmd.sh
vaishali@DESKTOP-LHT8EKU: ~$ ./avgcmd.sh
./avgcmd.sh: line 8: sum / count: division by 0 (error token is "count")
Average:
vaishali@DESKTOP-LHT8EKU: ~$ ./avgcmd.sh 10 20 30 40
Average: 25
vaishali@DESKTOP-LHT8EKU: ~$
```

4. Write a shell program to exchange the values of two variables

STEP 1 : Create a file named- swap.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type chmod +x swap.sh

STEP 4: For output type ./ swap.sh

```
#!/bin/bash

echo "Enter first number:"

read a

echo "Enter second number:"

read b

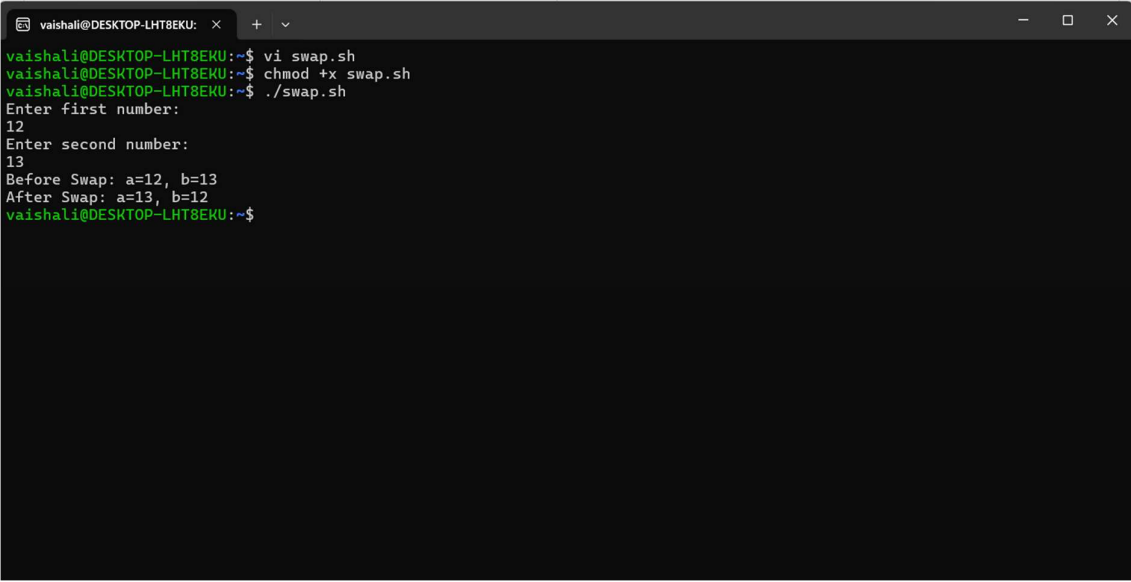
echo "Before Swap: a=$a, b=$b"

temp=$a

a=$b

b=$temp

echo "After Swap: a=$a, b=$b"
```

A terminal window titled 'vaishali@DESKTOP-LHT8EQU' with standard window controls. The terminal shows the following commands and output:

```
vaishali@DESKTOP-LHT8EQU:~$ vi swap.sh
vaishali@DESKTOP-LHT8EQU:~$ chmod +x swap.sh
vaishali@DESKTOP-LHT8EQU:~$ ./swap.sh
Enter first number:
12
Enter second number:
13
Before Swap: a=12, b=13
After Swap: a=13, b=12
vaishali@DESKTOP-LHT8EQU:~$
```



```
i=1
while [ $i -le 10 ]
do
    echo $i
    i=$((i + 1))
done
```

```
echo "" # Blank line for separation
```

```
o# Simulating do-while loop
```

```
echo "Numbers from 1 to 10 using simulated do-while loop:"
```

```
j=1
while true
do
    echo $j
    j=$((j + 1))
    if [ $j -gt 10 ]; then
        break
    fi
done
```

```
vaishali@DESKTOP-LHT8EQU: x + v
#!/bin/bash

# Using while loop
echo "Numbers from 1 to 10 using while loop:"
i=1
while [ $i -le 10 ]
do
    echo $i
    i=$((i + 1))
done

echo "" # Blank line for separation

# Simulating do-while loop
echo "Numbers from 1 to 10 using simulated do-while loop:"
j=1
while true
do
    echo $j
    j=$((j + 1))
    if [ $j -gt 10 ]; then
        break
    fi
done

~
~
~
~
:WQ
```

```
vaishali@DESKTOP-LHT8EQU: x + v
vaishali@DESKTOP-LHT8EQU:~$ vi dowhileloop.sh
vaishali@DESKTOP-LHT8EQU:~$ chmod +x dowhileloop.sh
vaishali@DESKTOP-LHT8EQU:~$ ./dowhileloop.sh
Numbers from 1 to 10 using while loop:
1
2
3
4
5
6
7
8
9
10

Numbers from 1 to 10 using simulated do-while loop:
1
2
3
4
5
6
7
8
9
10
vaishali@DESKTOP-LHT8EQU:~$
```

6. Write a shell program to Print Numbers 1 to 10 using for loop.

STEP 1 : Create a file named- vi forloopp.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type chmod +x forloopp.sh

STEP 4: For output type ./ forloopp.sh

```
#!/bin/bash
```

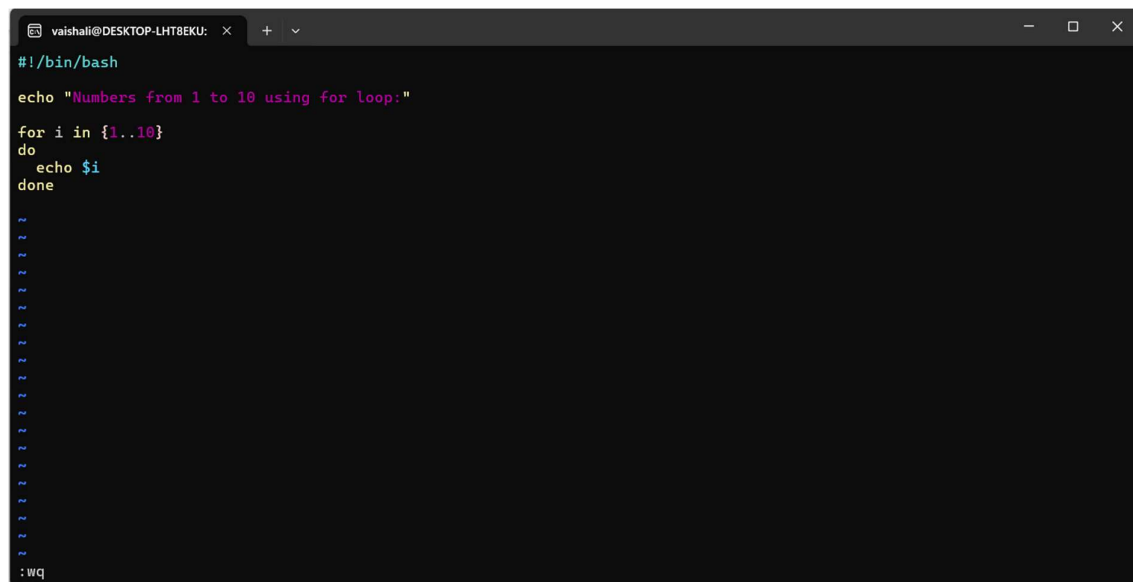
```
echo "Numbers from 1 to 10 using for loop:"
```

```
for i in {1..10}
```

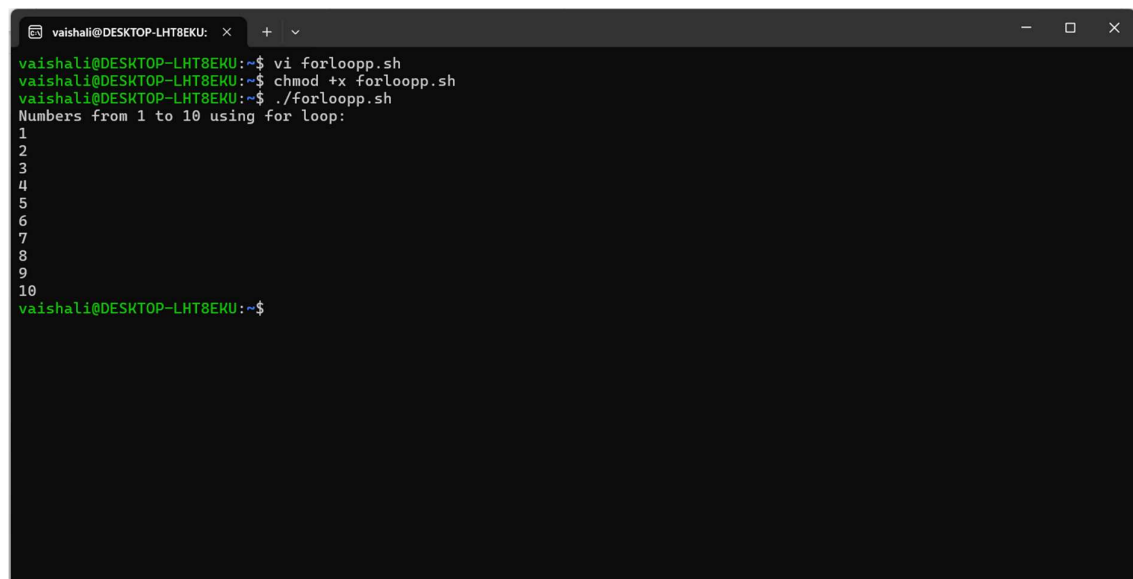
```
do
```

```
    echo $i
```

```
done
```

A terminal window titled 'vaishali@DESKTOP-LHT8EQU' with a dark background. It displays the script content:

```
#!/bin/bash
echo "Numbers from 1 to 10 using for loop:"
for i in {1..10}
do
    echo $i
done
```

 The cursor is at the end of the 'done' line. There are several tilde characters (~) visible below the script, likely representing line continuation or a list of files.A terminal window titled 'vaishali@DESKTOP-LHT8EQU' with a dark background. It shows the execution of the script:

```
vaishali@DESKTOP-LHT8EQU:~$ vi forloopp.sh
vaishali@DESKTOP-LHT8EQU:~$ chmod +x forloopp.sh
vaishali@DESKTOP-LHT8EQU:~$ ./forloopp.sh
Numbers from 1 to 10 using for loop:
1
2
3
4
5
6
7
8
9
10
vaishali@DESKTOP-LHT8EQU:~$
```

7. Write a shell script to display the digits which are in odd position in a given 5-digit number.

STEP 1 : Create a file named- vi cuttcomand.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type `chmod +x cuttcommand.sh`

STEP 4: For output type ./ cuttcomand.sh

Cut command Extracts the **1st, 3rd 5th character** of the input.

```
vaishali@DESKTOP-LHT8EKU: x + v
#!/bin/bash
echo "Enter a 5-digit number:"
read num

digit1=$(echo $num | cut -c1)
digit3=$(echo $num | cut -c3)
digit5=$(echo $num | cut -c5)

echo "Digits in odd positions: $digit1 $digit3 $digit5"

:WQ
```

```
vaishali@DESKTOP-LHT8EKU: ~$ vi cuttcomand.sh
vaishali@DESKTOP-LHT8EKU: ~$ chmod +x cuttcomand.sh
vaishali@DESKTOP-LHT8EKU: ~$ ./cuttcomand.sh
Enter a 5-digit number:
14567
Digits in odd positions: 1 5 7
vaishali@DESKTOP-LHT8EKU: ~$
```

8. Write a shell program to search for a given number from the list of numbers provided using binary search method.

STEP 1 : Create a file named- vi binarysearch.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type chmod +x binarysearch.sh

STEP 4: For output type ./ binarysearch.sh

```
#!/bin/bash

echo "Enter sorted numbers (space separated):"

read -a arr

echo "Enter number to search:"

read key

low=0

high=$(( ${#arr[@]} - 1))

while [ $low -le $high ]

do

    mid=$(( (low + high) / 2))
```



```
if [ ${arr[$mid]} -eq $key ]; then

    echo "Found at position ${((mid+1))}"

    exit 0

elif [ ${arr[$mid]} -lt $key ]; then

    low=$((mid + 1))

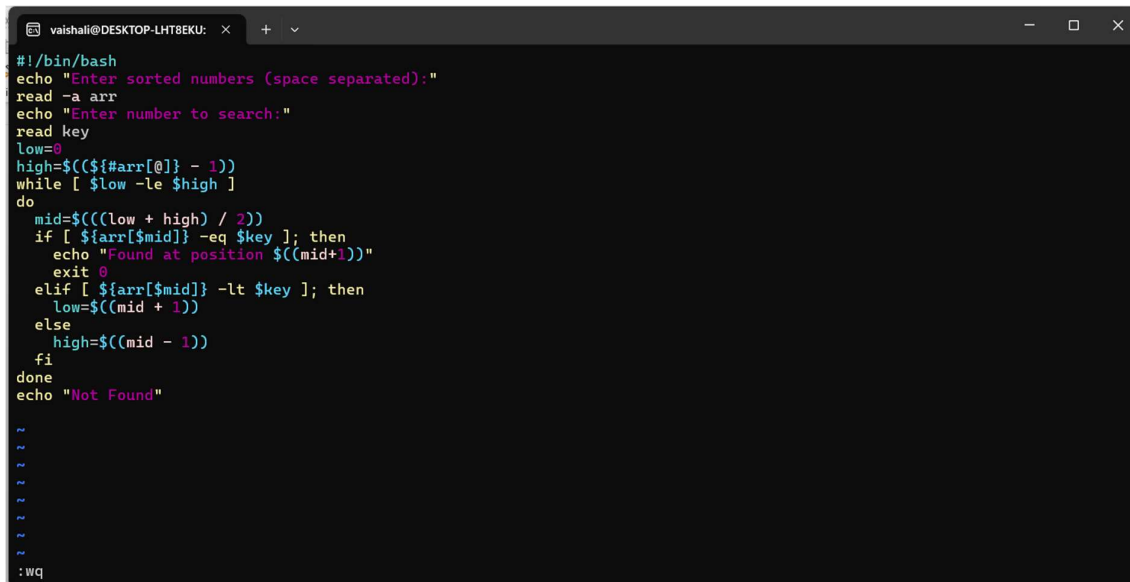
else

    high=$((mid - 1))

fi

done

echo "Not Found"
```

A terminal window titled 'vaishali@DESKTOP-LHT8EQU' with standard window controls. It contains a bash script for binary search. The script prompts the user to enter sorted numbers and a search key. It then enters a while loop that repeatedly calculates the middle index and compares the middle element with the search key. If found, it prints the position and exits. If not found, it prints 'Not Found'. The terminal shows the script's execution with some output lines highlighted in pink.

```
#!/bin/bash
echo "Enter sorted numbers (space separated):"
read -a arr
echo "Enter number to search:"
read key
low=0
high=$(( ${#arr[@]} - 1 ))
while [ $low -le $high ]
do
    mid=$(( (low + high) / 2 ))
    if [ ${arr[$mid]} -eq $key ]; then
        echo "Found at position ${((mid+1))}"
        exit 0
    elif [ ${arr[$mid]} -lt $key ]; then
        low=$((mid + 1))
    else
        high=$((mid - 1))
    fi
done
echo "Not Found"

~
~
~
~
~
~
~
:wq
```

```
vaishali@DESKTOP-LHT8EKU: ~$ vi binarysearch.sh
vaishali@DESKTOP-LHT8EKU: ~$ vi binarysearch.sh
vaishali@DESKTOP-LHT8EKU: ~$ chmod +x binarysearch.sh
vaishali@DESKTOP-LHT8EKU: ~$ ./binarysearch.sh
Enter sorted numbers (space separated):
10 12 13 14 15 16 18 19 20
Enter number to search:
14
Found at position 4
vaishali@DESKTOP-LHT8EKU: ~$
```

9. Write a shell program to concatenate two strings and find the length of the resultant string

STEP 1 : Create a file named- vi concatlen.sh

STEP 2: Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3: To execute the file type chmod +x concatlen.sh

STEP 4: For output type ./ concatlen.sh

```
#!/bin/bash
echo "Enter first string:"
read str1
echo "Enter second string:"
read str2
concat="$str1$str2"
echo "Concatenated string: $concat"
echo "Length: ${#concat}"
```



```
#!/bin/bash
echo "Enter three numbers:"
read a b c
if [ $a -le $b ] && [ $a -le $c ]; then
    echo "$a is smallest"
elif [ $b -le $a ] && [ $b -le $c ]; then
    echo "$b is smallest"
else
    echo "$c is smallest"
fi
```

:wq

```
vaishali@DESKTOP-LHT8EKU: ~$ vi smallest.sh
vaishali@DESKTOP-LHT8EKU: ~$ chmod +x smallest.sh
vaishali@DESKTOP-LHT8EKU: ~$ ./smallest.sh
Enter three numbers:
10 14 9
9 is smallest
vaishali@DESKTOP-LHT8EKU: ~$
```

11. Write a shell program to count number of words, characters, white spaces and special symbols in a given text

STEP 1 : Create a file named- vi countall.sh

STEP 2 : Press I, Write the code and then, Press Esc , then :wq, and hit Enter to save and exit.(if force quit then :q!).

STEP 3 : To execute the file type chmod +x countall.sh

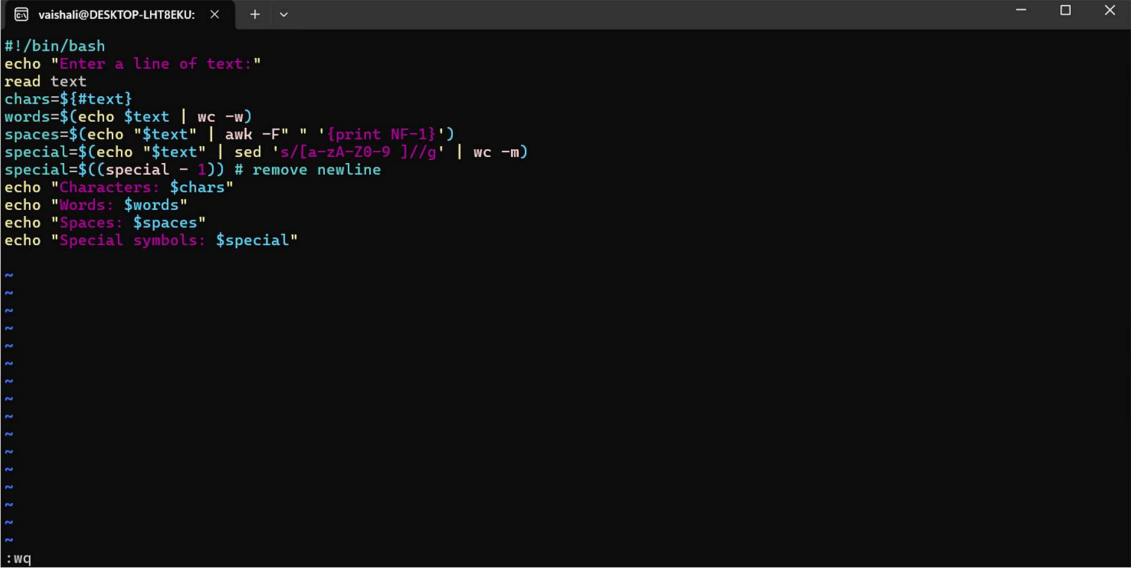
STEP 4 : For output type ./ countall.sh

```
#!/bin/bash
echo "Enter a line of text:"
read text
chars=${#text}
words=$(echo $text | wc -w)
spaces=$(echo "$text" | awk -F" " '{print NF-1}')
special=$(echo "$text" | sed 's/[a-zA-Z0-9 ]//g' | wc -m)
special=$((special - 1)) # remove newline
echo "Characters: $chars"
```

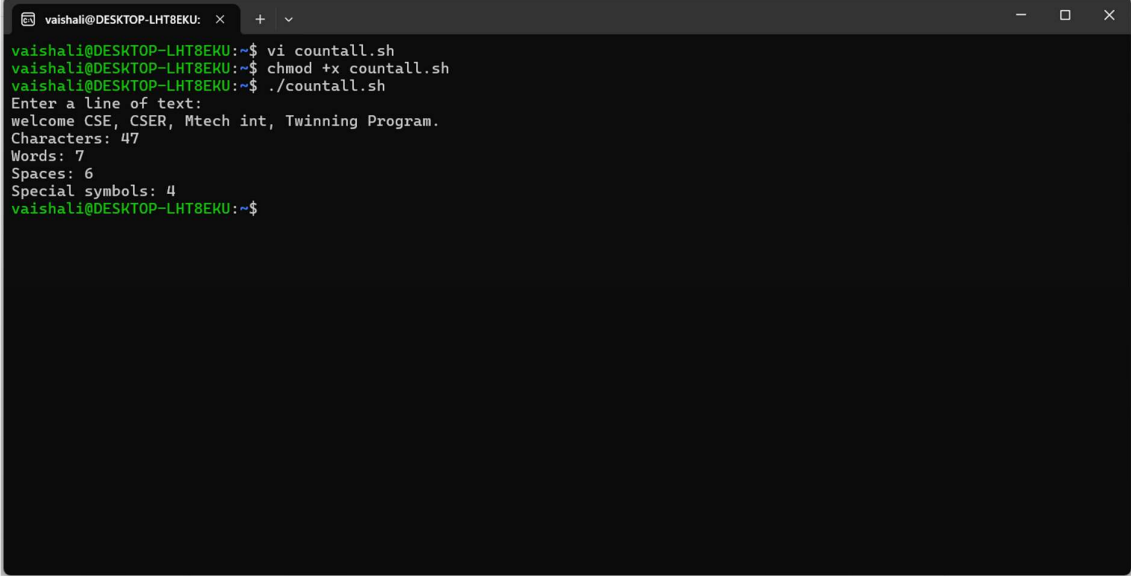
```
echo "Words: $words"

echo "Spaces: $spaces"

echo "Special symbols: $special"
```

A terminal window titled 'vaishali@DESKTOP-LHT8EQU' with a dark background. It displays the source code of a shell script. The script prompts the user to enter a line of text, then calculates and prints the number of characters, words, spaces, and special symbols. The script uses various command-line utilities like 'wc', 'awk', 'sed', and 'tr'.

```
#!/bin/bash
echo "Enter a line of text:"
read text
chars=${#text}
words=$(echo $text | wc -w)
spaces=$(echo "$text" | awk -F" " '{print NF-1}')
special=$(echo "$text" | sed 's/[a-zA-Z0-9 ]//g' | wc -m)
special=$((special - 1)) # remove newline
echo "Characters: $chars"
echo "Words: $words"
echo "Spaces: $spaces"
echo "Special symbols: $special"
```

A terminal window titled 'vaishali@DESKTOP-LHT8EQU' with a dark background. It shows the execution of the script 'countall.sh'. The user enters the text 'welcome CSE, CSER, Mtech int, Twinning Program.', and the script outputs the counts for characters, words, spaces, and special symbols.

```
vaishali@DESKTOP-LHT8EQU:~$ vi countall.sh
vaishali@DESKTOP-LHT8EQU:~$ chmod +x countall.sh
vaishali@DESKTOP-LHT8EQU:~$ ./countall.sh
Enter a line of text:
welcome CSE, CSER, Mtech int, Twinning Program.
Characters: 47
Words: 7
Spaces: 6
Special symbols: 4
vaishali@DESKTOP-LHT8EQU:~$
```